Dependent Type Theory

Charles Averill

The University of Texas at Dallas

October 22nd, 2026



Where System $F\omega$ Fails

- System $F\omega$ is pretty good, forms the basis of OCaml, Haskell, Rust, C++, Swift type systems
- Point of type systems is "well-typed programs can't 'go wrong' "
- But. . . they still can!

```
let hd (l : 'a list) : 'a =
  match l with
  | [] -> failwith "can't take hd of []"
  | h :: _ -> h
```

Where System $F\omega$ Fails

What if we could prevent such errors at compile time with a new classification of types?

```
let hd (l : 'a (n + 1) list) : 'a =
  match l with
  | h :: _ -> h
```

In this sample, the type of `hd`'s input argument *depends* on the computation value `n + 1` to ensure that the list is non-empty.

Lambda Calculus Refresher

Syntax

$$e := v \mid \lambda v.\, e \mid e_1\, e_2$$

Semantics

$$\frac{}{(\lambda v.\, e_1)e_2 \;\Downarrow\; e_1[e_2/v]}(\rightarrow_\beta)$$

$$\frac{P \qquad P \implies Q}{Q}$$

$$(\lambda x.x)(\lambda y.y) \rightarrow_\beta \lambda y.y$$

$$(\lambda x.\lambda y.x)(\lambda a.a)(\lambda b.b) \rightarrow_\beta (\lambda y.(\lambda a.a))(\lambda b.b) \rightarrow_\beta \lambda a.a$$

$$(\lambda f.(\lambda x.f\,x\,x)(\lambda x.f\,x\,x))(\lambda f.(\lambda x.f\,x\,x)(\lambda x.f\,x\,x)) \rightarrow_\beta ?$$

$$\texttt{succ [1,2,3]} \rightarrow_\beta ?$$

System $F\omega$ Refresher

Syntax

$$e := () \mid v \mid \lambda v : T.\, e \mid e_1\, e_2$$

$$T := \texttt{unit} \mid \texttt{void} \mid T_1 \to T_2$$

Typing Semantics

$$\frac{x : \sigma \in \Gamma}{\Gamma \vdash x : \sigma} \qquad \frac{\Gamma, x : \sigma \vdash e : \tau}{\Gamma \vdash (\lambda x : \sigma.\, e) : (\sigma \to \tau)} \qquad \frac{\Gamma \vdash e_1 : \sigma \to \tau \quad \Gamma \vdash e_2 : \sigma}{\Gamma \vdash e_1\, e_2 : \tau}\ (\lambda_\to)$$

Γ: *typing context*, function that maps idents to types

$\Gamma \vdash e : \tau$: "in context Γ, e has type τ"

$$\lambda x : \texttt{int}.\lambda y : \texttt{int}.\, x + y$$

$$(\lambda s : \texttt{str}.\, \texttt{"Hello, "} + s + \texttt{"!"})\, \texttt{"World"}$$

System $F\omega$ Refresher

Syntax

$$e := () \mid v \mid \lambda v : T.\, e \mid e_1\, e_2 \mid \Lambda\alpha.\, e$$

$$T := \texttt{unit} \mid \texttt{void} \mid T_1 \to T_2 \mid \alpha \mid \forall\alpha.\, T$$

Typing Semantics

$$\frac{x : \sigma \in \Gamma}{\Gamma \vdash x : \sigma} \qquad \frac{\Gamma, x : \sigma \vdash e : \tau}{\Gamma \vdash (\lambda x : \sigma.\, e) : (\sigma \to \tau)} \qquad \frac{\Gamma \vdash e_1 : \sigma \to \tau \quad \Gamma \vdash e_2 : \sigma}{\Gamma \vdash e_1\, e_2 : \tau} \; (\lambda_\to)$$

$$\frac{\Gamma, \alpha \vdash e : \sigma}{\Gamma \vdash \Lambda\alpha.\, e : \forall\alpha.\, \sigma} \qquad \frac{\Gamma \vdash e : \forall\alpha.\, \sigma}{\Gamma \vdash e\,[T] : \sigma[T/\alpha]} \; (\lambda_2)$$

$$\textsc{Cons} : \forall\alpha.\, \alpha \to \alpha\ \texttt{list} \to \alpha\ \texttt{list} := \Lambda\alpha.\, \lambda hd : \alpha . \lambda tl : \alpha\ \texttt{list}.\, (hd,\, tl)$$

System $F\omega$ Refresher

Syntax

$$e := () \mid v \mid \lambda v : T.\ e \mid e_1\ e_2 \mid \Lambda\alpha :: K.\ e \mid e\ [T]$$

$$T := \texttt{unit} \mid \texttt{void} \mid T_1 \to T_2 \mid \alpha \mid \forall\alpha :: K.\ T \mid \lambda\alpha :: K.\ T \mid T_1\ T_2$$

$$K := * \mid K_1 \to K_2$$

Typing Semantics (assuming type equivalence via β-reduction)

$$\frac{x : \sigma \in \Gamma}{\Gamma \vdash x : \sigma} \qquad \frac{\Gamma, x : \sigma \vdash e : \tau}{\Gamma \vdash (\lambda x : \sigma.\ e) : (\sigma \to \tau)} \qquad \frac{\Gamma \vdash e_1 : \sigma \to \tau \quad \Gamma \vdash e_2 : \sigma}{\Gamma \vdash e_1\ e_2 : \tau}\ (\lambda_\to)$$

$$\frac{\Gamma, \alpha \vdash e : \sigma}{\Gamma \vdash \Lambda\alpha.\ e : \forall\alpha.\ \sigma} \qquad \frac{\Gamma \vdash e : \forall\alpha.\ \sigma \quad \Gamma \vdash T :: \kappa}{\Gamma \vdash e\ [T] : \sigma[T/\alpha]}\ (\lambda_2) \qquad \frac{\alpha :: \kappa \in \Gamma}{\Gamma \vdash \alpha :: \kappa} \qquad \frac{t \in \lambda_2}{\Gamma \vdash t :: *}$$

$$\frac{\Gamma, x :: \kappa_1 \vdash t :: \kappa_2}{\Gamma \vdash (\lambda x :: \kappa_1.\ t) :: (\kappa_1 \to \kappa_2)} \qquad \frac{\Gamma \vdash t_1 :: \kappa_1 \to \kappa_2 \qquad \Gamma \vdash t_2 :: \kappa_1}{\Gamma \vdash t_1\ t_2 :: \kappa_2}\ (\lambda_{\underline{\omega}})$$

System $F\omega$ Refresher

$$\textsc{Pair} :: * \to * \to * :=$$
$$\lambda A :: *. \lambda B :: *. \forall F. (A \to B \to F) \to F$$
$$\texttt{tuple}: \forall A. \forall B.\ A \to B \to \textsc{Pair}\ A\ B :=$$
$$\Lambda A. \Lambda B. \lambda fst : A. \lambda snd : B. \Lambda F. \lambda f : (A \to B \to F). f\ fst\ snd$$
$$\textsc{Sum} :: * \to * \to * :=$$
$$\lambda A :: *. \lambda B :: *. \forall F. (A \to F) \to (B \to F) \to F$$
$$\texttt{left}: \forall A. \forall B.\ A \to \textsc{Sum}\ A\ B :=$$
$$\Lambda A. \Lambda B. \lambda a : A. \Lambda F. \lambda f_1 : (A \to F). \lambda f_2 : (B \to F). f_1\ a$$
$$\texttt{right}: \forall A. \forall B.\ B \to \textsc{Sum}\ A\ B :=$$
$$\Lambda A. \Lambda B. \lambda b : B. \Lambda F. \lambda f_1 : (A \to F). \lambda f_2 : (B \to F). f_2\ b$$

Type Inhabitation

A type is *inhabited* iff there exists a term with that type.

Type	Inhabited?
$\texttt{unit}$	$()$
$\texttt{void}$	$\times$
$\texttt{unit} \to \texttt{unit}$	$\lambda x : \texttt{unit}.\ x$
$\texttt{unit} \to \texttt{void}$	$\times$
$\texttt{void} \to \texttt{unit}$	$\lambda x : \texttt{void}.\ ()$
$\texttt{void} \to \texttt{void}$	$\lambda x : \texttt{void}.\ x$
$\forall \alpha :: *.\ \alpha \to \alpha$	$\Lambda \alpha :: *.\ \lambda x : \alpha.\ x$
$\forall \alpha :: *.\ \alpha$	$\times$
$\forall \alpha :: *.\ \forall \beta :: *.\ \alpha \to \beta$	$\times$
$(\lambda \beta :: *.\ \beta \to \beta)\ \texttt{unit}$	$\lambda x : \texttt{unit}.\ x$
$\forall F :: * \to *.\ \forall \alpha :: *.\ F\ \alpha \to F\ \alpha$	$\Lambda F :: * \to *.\ \Lambda \alpha :: *.\ \lambda x : F\ \alpha.\ x$

Type Inhabitation

We can fill out this relation with some added standard types, like tuples:

Type	Inhabited
`unit`	True
`void`	False
$\texttt{void} \times \texttt{void}$	False
$\texttt{void} \times \texttt{unit}$	False
$\texttt{unit} \times \texttt{void}$	False
$\texttt{unit} \times \texttt{unit}$	True
$\texttt{void} \to \texttt{void}$	True
$\texttt{void} \to \texttt{unit}$	True
$\texttt{unit} \to \texttt{void}$	False
$\texttt{unit} \to \texttt{unit}$	True
$\forall \alpha :: *.\ \alpha$	False
$\forall \alpha :: *.\ \alpha \to \alpha$	True

Type Inhabitation

We can fill out this relation with some added standard types, like tuples:

Logical Expression	Truth Value
T	True
F	False
$F \wedge F$	False
$F \wedge T$	False
$T \wedge F$	False
$T \wedge T$	True
$F \to F$	True
$F \to T$	True
$T \to F$	False
$T \to T$	True
$\forall P.\ P$	False
$\forall P.\ P \to P$	True

Break! Any Questions?

Types as Propositions

It turns out that System $F\omega$ corresponds with Higher-Order Propositional Logic (HOL)! $F\omega$ types are propositions, and its values are proofs of propositions.

```
let modus_ponens : P -> (P -> Q) -> Q =
  fun (p : P) (f : P -> Q) ->
    f p
```

$$\frac{P \quad P \implies Q}{Q}(\text{MP})$$

What do we get with higher-order propositional logic?

- Propositions that talk about other propositions (reasoning about logical connectives):

$$\textsc{Symmetric}(F) \coloneqq \forall P\, Q,\ F\, P\, Q \implies F\, Q\, P$$

- Encodings of impredicative relations and data structures:

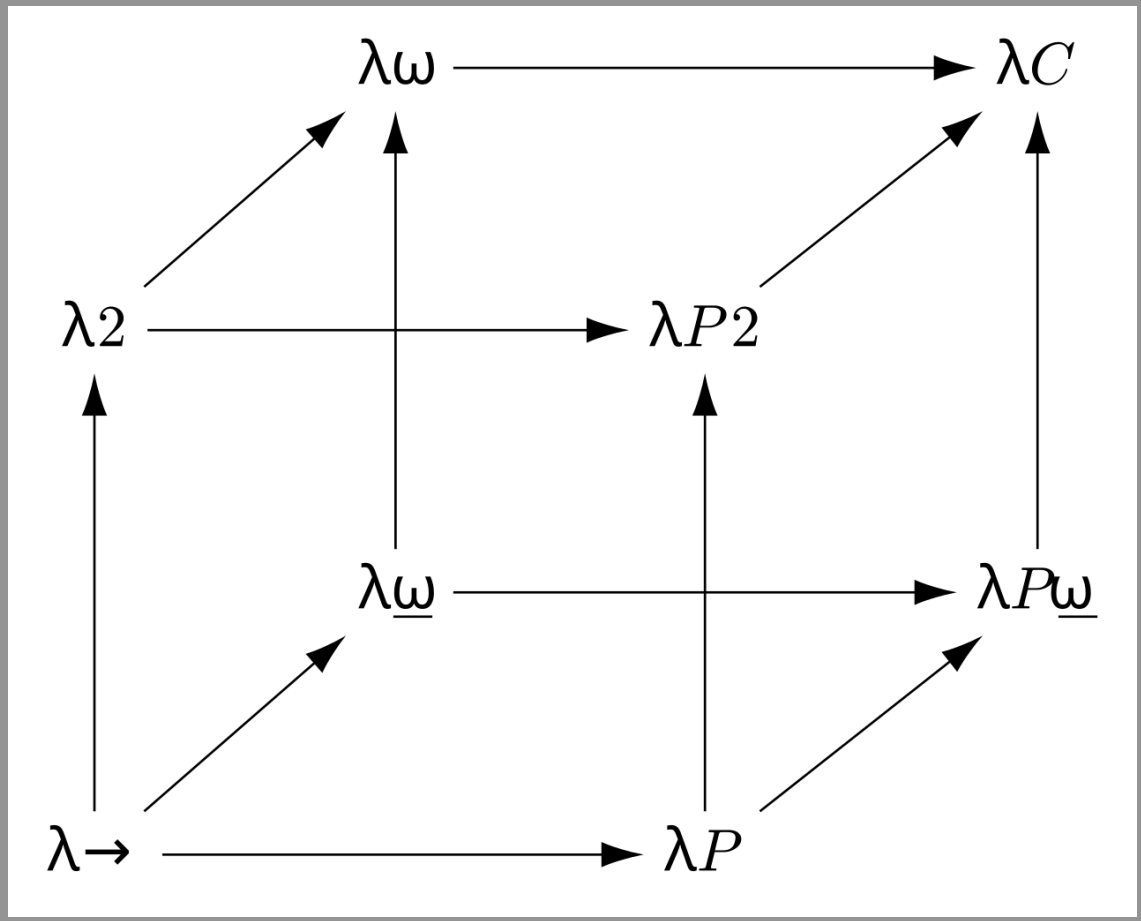
$$\textsc{Eq}(P,\ Q) \coloneqq \forall R,\ R\, P \implies R\, Q$$

Where it Stops

- HOL is pretty nice (see Isabelle/HOL), but has limitations
- Can't even define predicates over values, like `5 is odd`
- HOL propositions can only take *other propositions* as parameters
- Not even induction is native to HOL!

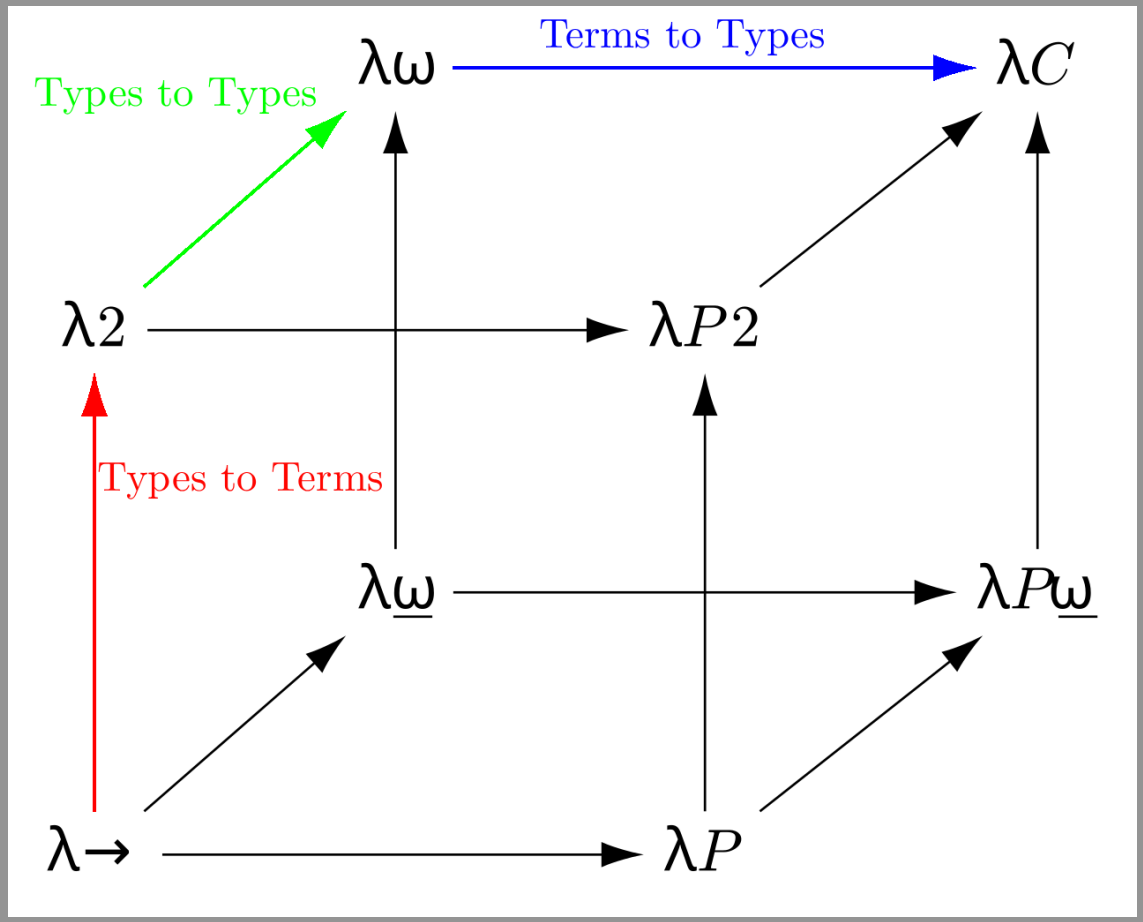
Back to PL

The problem is related to the lambda calculi typing semantics!



Back to PL

The problem is related to the lambda calculi typing semantics!



λC

Syntax

$$e := \lambda_\omega$$
$$T := \lambda_\omega \mid \Pi v : T_1 . T_2$$
$$K := \lambda_\omega \mid \Pi v : T . K$$

Typing Semantics (plus λ_ω)

$$\frac{\Gamma \vdash T_1 :: * \qquad \Gamma, x : T_1 \vdash T_2 :: *}{\Gamma \vdash \Pi x : T_1 . T_2 :: *} \qquad \frac{\Gamma \vdash S :: \Pi x : T . K \qquad \Gamma \vdash t : T}{\Gamma \vdash S\, t :: K[t/x]}$$

$$\frac{\Gamma \vdash S :: * \qquad \Gamma, x : S \vdash t : T}{\Gamma \vdash (\lambda x : S . t) : \Pi x : S . T} \qquad \frac{\Gamma \vdash t_1 : \Pi x : S . T \qquad \Gamma \vdash t_2 : S}{\Gamma \vdash t_1\, t_2 : T[t_2/x]}$$

$$\frac{\Gamma \vdash T :: * \qquad \Gamma, x : T \vdash S :: K}{\Gamma \vdash (\lambda x : T . S) :: \Pi x : T . K} \qquad \frac{\Gamma \vdash T :: * \qquad \Gamma, x : T \vdash K}{\Gamma \vdash \Pi x : T . K} \; (\lambda C)$$

Dependent Vectors

$$\textsc{Vec} :: * \to \Pi n : \texttt{nat}. * :=$$
$$\lambda T :: *. \lambda n : \texttt{nat}. n = 0 \; ? \; \texttt{unit} : \textsc{Pair}\; T\; (\textsc{Vec}\; T\; (n-1))$$
$$\texttt{nil} : \forall T :: *. \textsc{Vec}\; T\; 0 := \Lambda T :: *. ()$$
$$\texttt{cons} : \forall T :: *. \Pi n : \texttt{nat}. T \to \textsc{Vec}\; T\; n \to \textsc{Vec}\; T\; (n+1) :=$$
$$\Lambda T :: *. \lambda n : \texttt{nat}. \lambda hd : T. \lambda tl : \textsc{Vec}\; T\; n. \texttt{tuple}\; [T]\; [\textsc{Vec}\; T\; n]\; hd\; tl$$
$$\texttt{hd} : \forall T :: *. \Pi n : nat. \textsc{Vec}\; T\; (n+1) \to T :=$$
$$\texttt{fst}\; [T]\; [\textsc{Vec}\; T\; n]$$
$$\texttt{tl} : \forall T :: *. \Pi n : nat. \textsc{Vec}\; T\; (n+1) \to \textsc{Vec}\; T\; n :=$$
$$\texttt{snd}\; [T]\; [\textsc{Vec}\; T\; n]$$